

LangChain & Agentic AI – From Basics to Advanced demo-15 Notes

=====

```
x = int(input("Enter First Number:"))  
y = int(input("Enter Second Number:"))
```

```
try:  
    print(x/y)  
except:  
    print('Exception Raised')
```

```
x = int(input("Enter First Number:"))  
y = int(input("Enter Second Number:"))
```

```
try:  
    print(x/y)  
except ZeroDivisionError:  
    print('Exception Raised')
```

```
-----  
x = int(input("Enter First Number:"))  
y = int(input("Enter Second Number:"))
```

```
try:  
    print(x/y)  
except ZeroDivisionError as e:  
    print('Exception Raised:',e)
```

```
-----  
from pydantic import BaseModel
```

```
class Student(BaseModel):  
    name:str  
    age:int  
    course:str
```

```
#student = Student("Durga",22,"Python") --->Invalid  
student = Student(name = "Durga",age = 22,course = "Python")  
print(student)
```

```
print(student.name)
print(student.age)
print(student.course)
print(type(student))
```

Validation?

Validation means checking whether data follows the rules defined by our schema.

Pydantic is responsible for = Define Schema + Validation

If validation succeeds then returns Pydantic class object.

If validation fails then it will raise `ValidationError`

eg:

```
from pydantic import BaseModel, ValidationError
```

```
class Student(BaseModel):
```

```
    name:str
```

```
    age:int
```

```
course:str
```

```
try:
```

```
    student = Student(name = "Durga",age = 22,course=10)
```

```
except ValidationError as e:
```

```
    print(e)
```

How to define Optional Fields:

Pydantic class can contain optional fields also. We can define as follows

```
course:str | None = None
```

```
class Student(BaseModel):
```

```
    name:str
```

```
    age:int
```

```
    course:str | None = None
```

name:str

age:int

These 2 are mandatory

course field is Optional

```
from pydantic import BaseModel, ValidationError
```

```
class Student(BaseModel):
```

```
    name:str
```

```
    age:int
```

```
    course:str | None = None
```

```
student = Student(name = "Durga",age = 22,course="Python")
```

```
print(student)
```

```
student = Student(name = "Durga",age = 22)
```

```
print(student)
```

```
E:\lc\classes>py test.py
```

```
name='Durga' age=22 course='Python'
```

```
name='Durga' age=22 course=None
```

```
=====
```

```
from pydantic import BaseModel, ValidationError
```

```
class Student(BaseModel):
```

```
    name:str
```

```
    age:int
```

```
    course:str | None = "LangChain"
```

```
student = Student(name = "Durga",age = 22)
```

```
print(student)
```

```
=====
```

```
name:str ==>Required field
```

```
name:str | None = None ==>Optional field with default  
value None
```

name:str | None = "Durga" =====>Optional field with default value Durga

name:str | None = 10 =====>ValidationError

=====

Note:

Dont make every field optional

Dont make every field required

Design according to our application requirement.

=====

What is Field():

Sometimes field names alone may not enough. We can provide description also.

name:str

name:str = Field(description = "Name of the Student")

Second one is more readable and provide more information to the LLM.

We can provide default value and description as follows:

```
name:str | None = Field(  
    default="Durga",  
    description = "Name of the Student"  
)
```

```
from pydantic import BaseModel, Field
```

```
class Student(BaseModel):  
    name:str | None = Field(default="Durga",  
description="Name of the student")  
    age:int = Field(description = 'Age of the student')  
    course:str | None = "LangChain"
```

```
student = Student(name = "Ravi",age = 22,course="Python")  
print(student)
```

```
student = Student(age = 22)  
print(student)
```